

Game Playing

Why do AI researchers study game playing?

1. It's a good reasoning problem, formal and nontrivial.
2. Direct comparison with humans and other computer programs is easy.

Types of games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships, blind tictactoe	bridge, poker, scrabble nuclear war

Two-player games

- A game formulated as a search problem:
 - **Initial state:** board position and turn
 - **Operators:** definition of legal moves
 - **Terminal state:** conditions for when game is over
 - **Utility function:** a numeric value that describes the outcome of the game.
e.g., -1, 0, 1 for loss, draw, win.

—ÖZYEĞİN—
—ÜNİVERSİTESİ—



We will consider games with 2 players
(Max and Min);
Max moves first.

Minimax

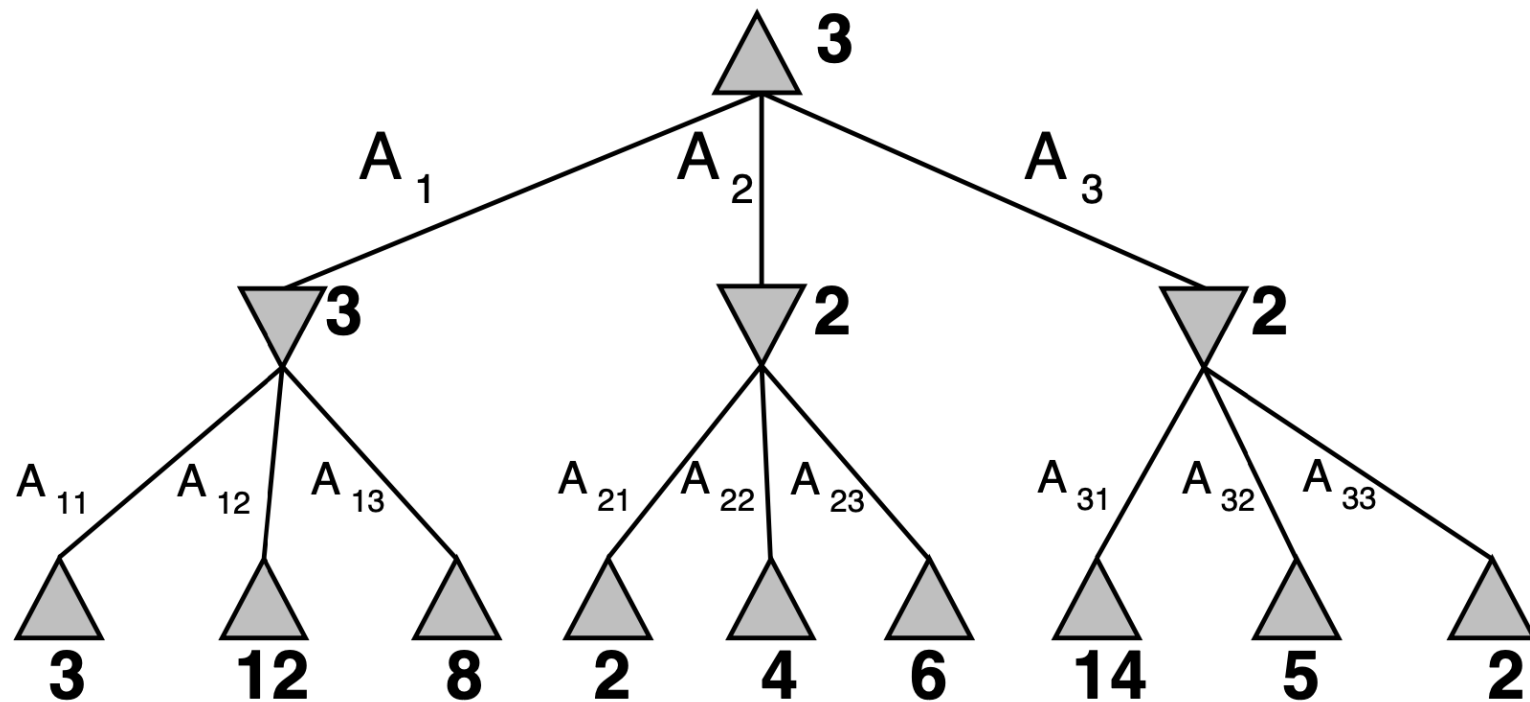
Perfect play for deterministic, perfect-information games

Idea: choose move to position with highest **minimax value**
= best achievable payoff against best play

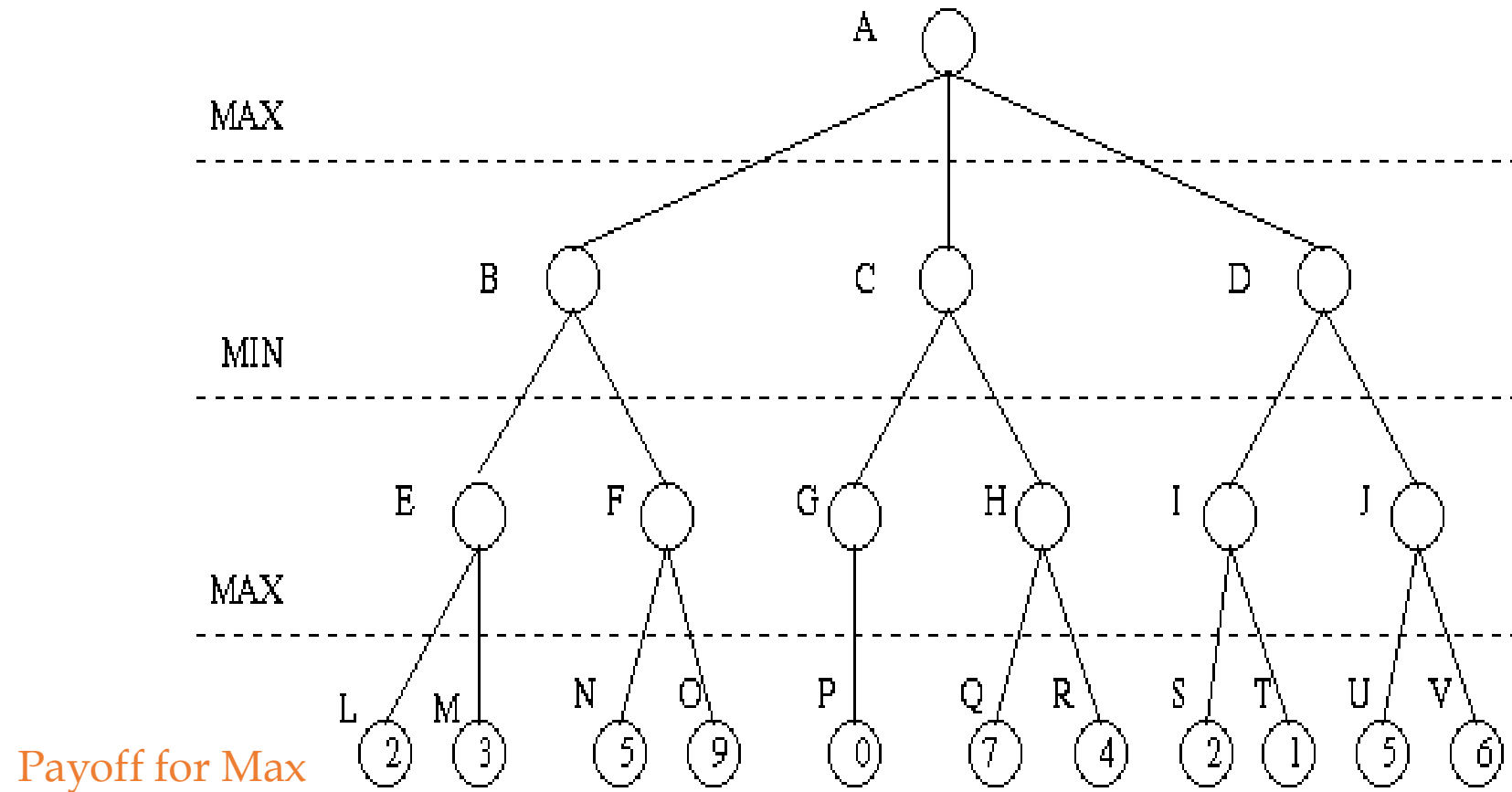
E.g., 2-ply game:

MAX

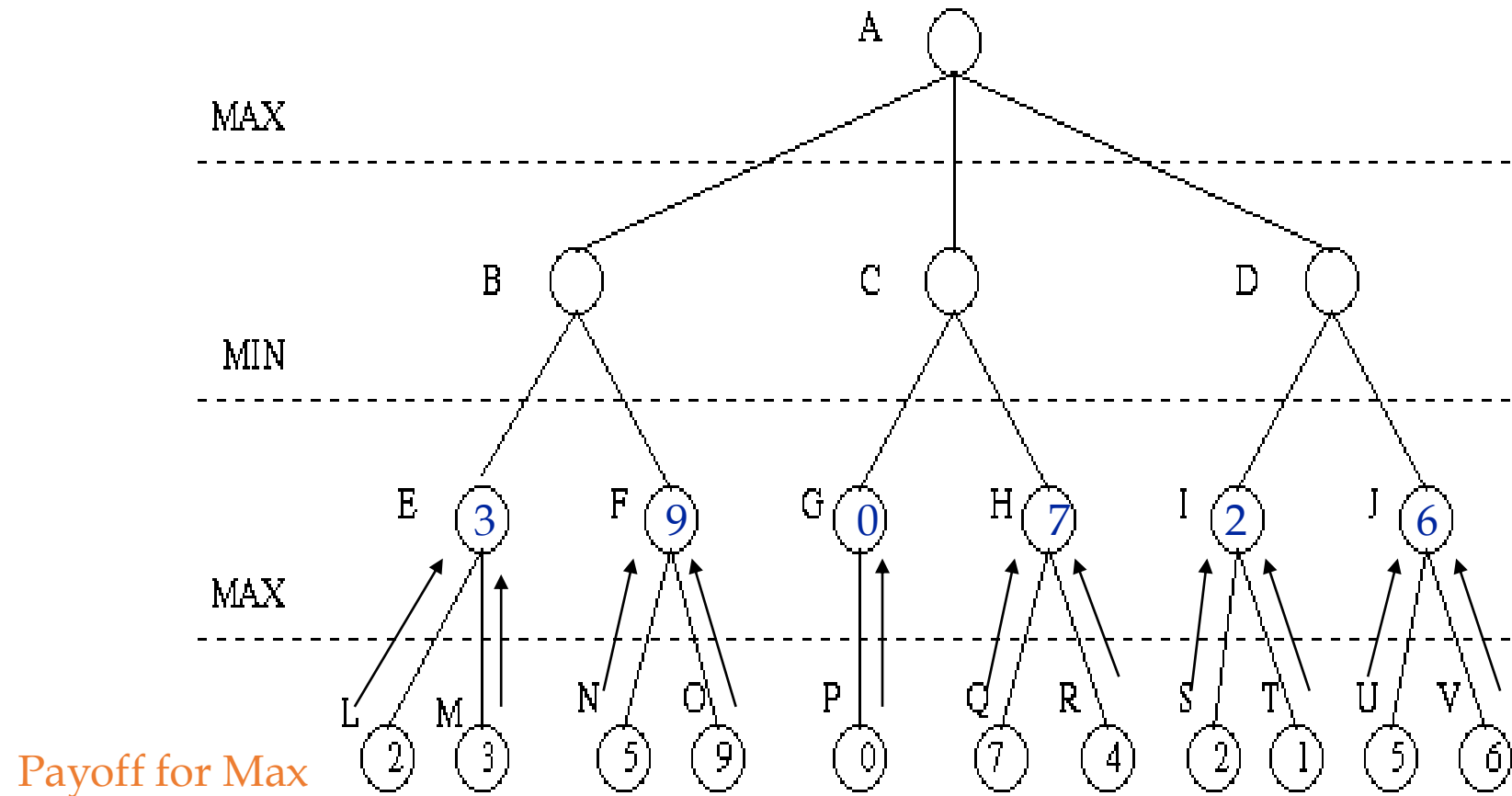
MIN



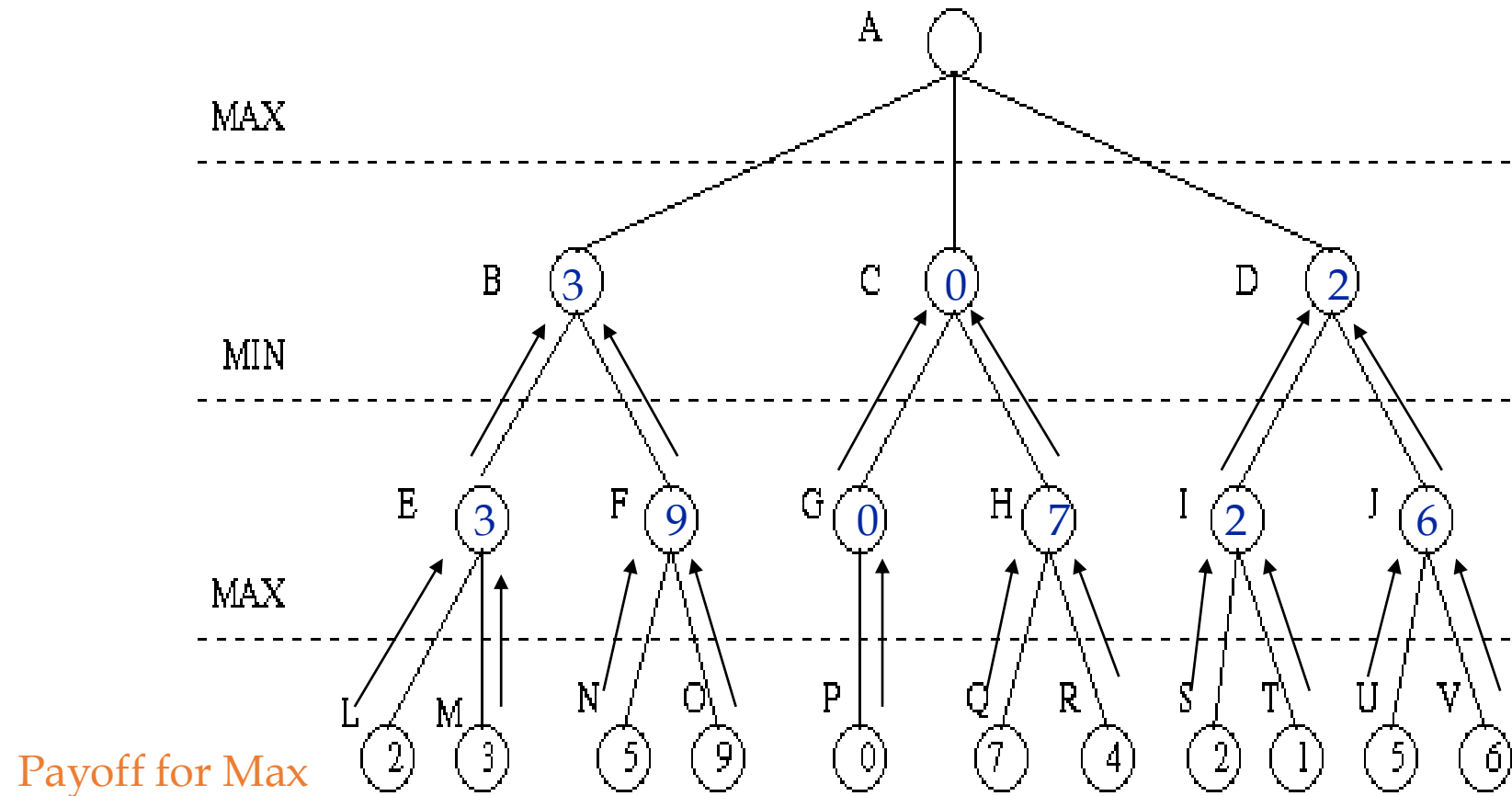
Minimax Algorithm



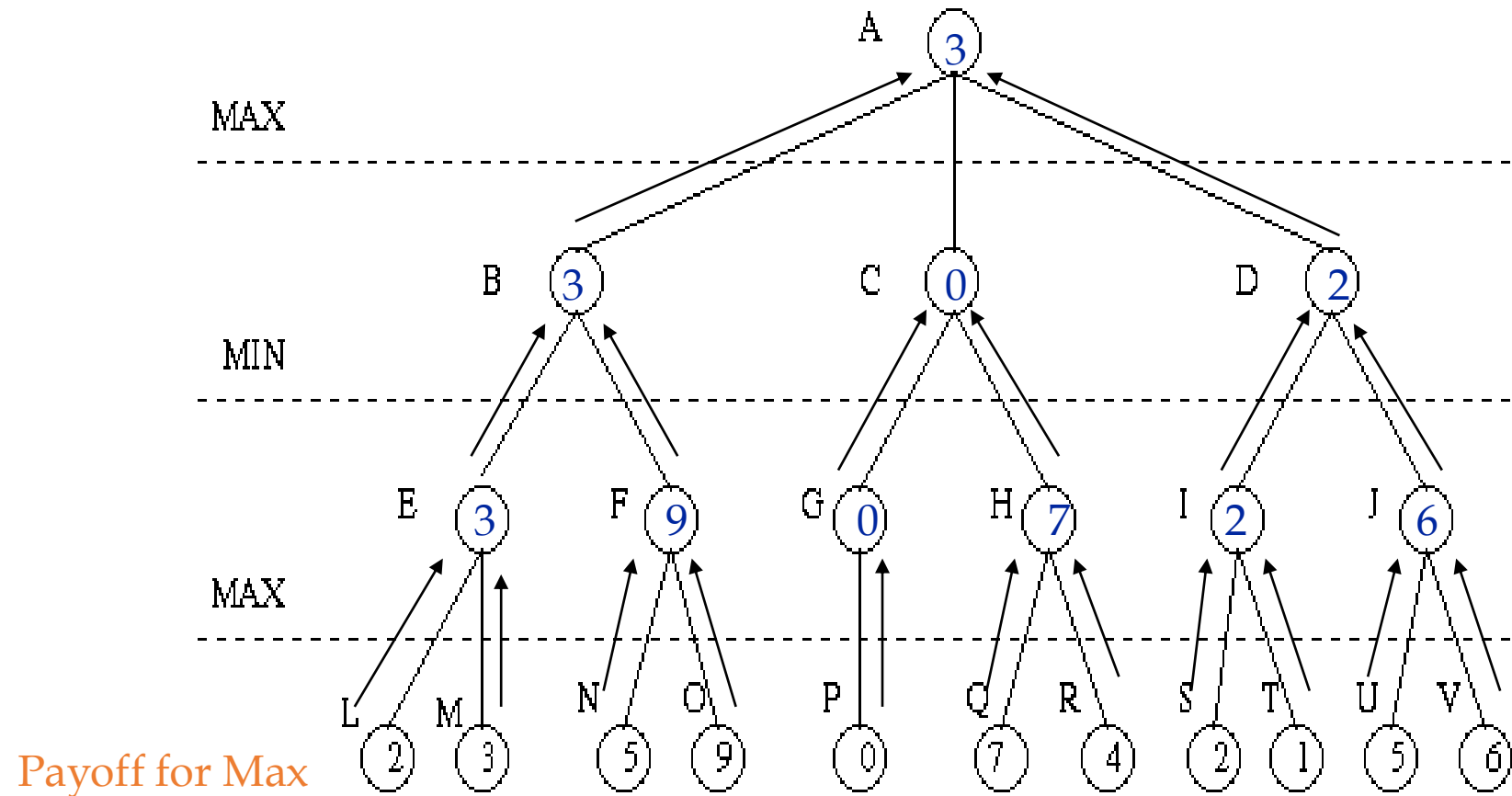
Minimax Algorithm (cont'd)



Minimax Algorithm (cont'd)



Minimax Algorithm (cont'd)



Minimax Algorithm

```
function MINIMAX-DECISION(state) returns an action  
  inputs: state, current state in game  
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))
```

```
function MAX-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$   
  return v
```

```
function MIN-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow \infty$   
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$   
  return v
```

Properties of Minimax

- Completeness: if tree is finite (chess has specific rules for this)
- Optimality: Yes, against an optimal opponent.
- Time complexity: $O(b^m)$
- Space complexity: $O(bm)$ (depth-first exploration)
- For chess, $b \approx 35$, $m \approx 100$ for “reasonable” games $\therefore$ exact solution completely infeasible
- But do we need to explore every path?

Move evaluation without complete search

- Complete search is too complex and impractical
- Evaluation function: evaluates value of state using **heuristics** and **cuts off search**

α - β pruning: search cutoff

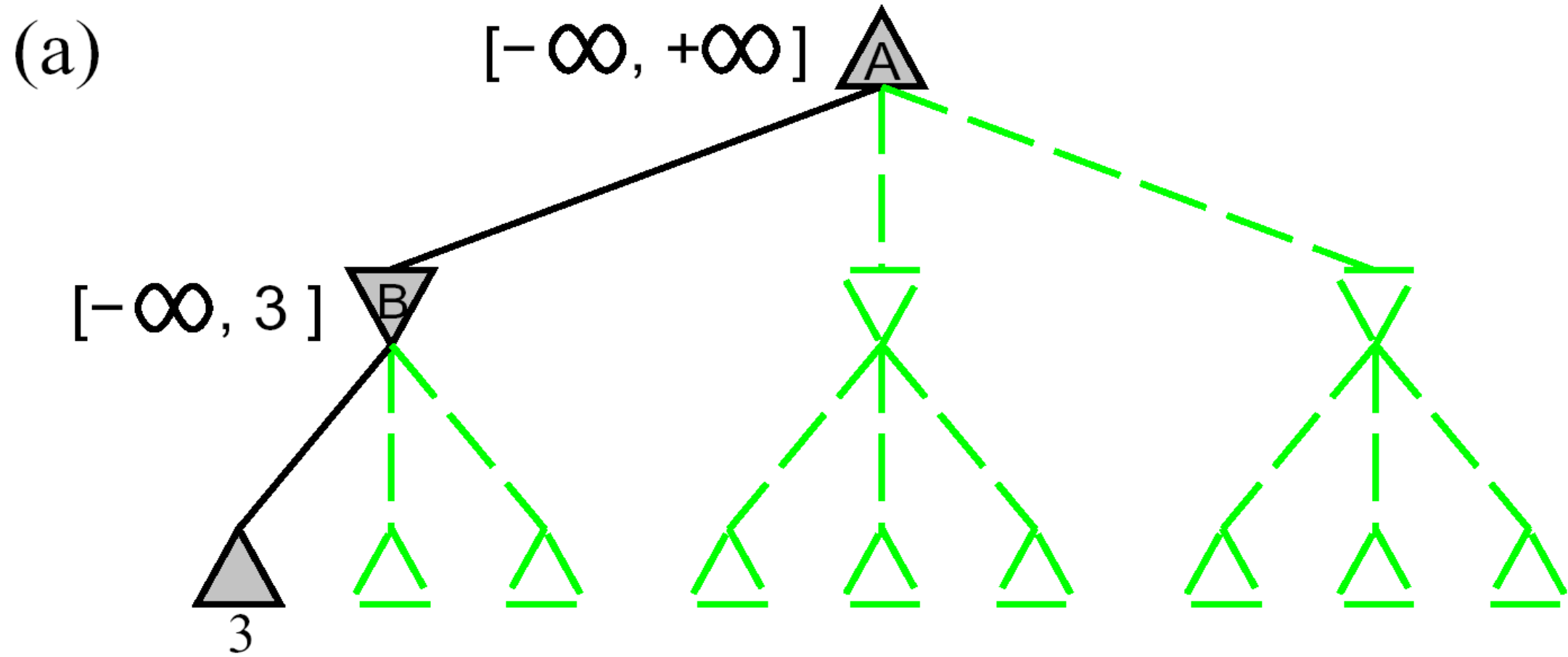
- **Pruning:** eliminating a branch of the search tree from consideration without exhaustive examination of each node
- **α - β pruning:** the basic idea is to prune portions of the search tree that cannot improve the utility value of the max or min node, by just considering the values of nodes seen so far.

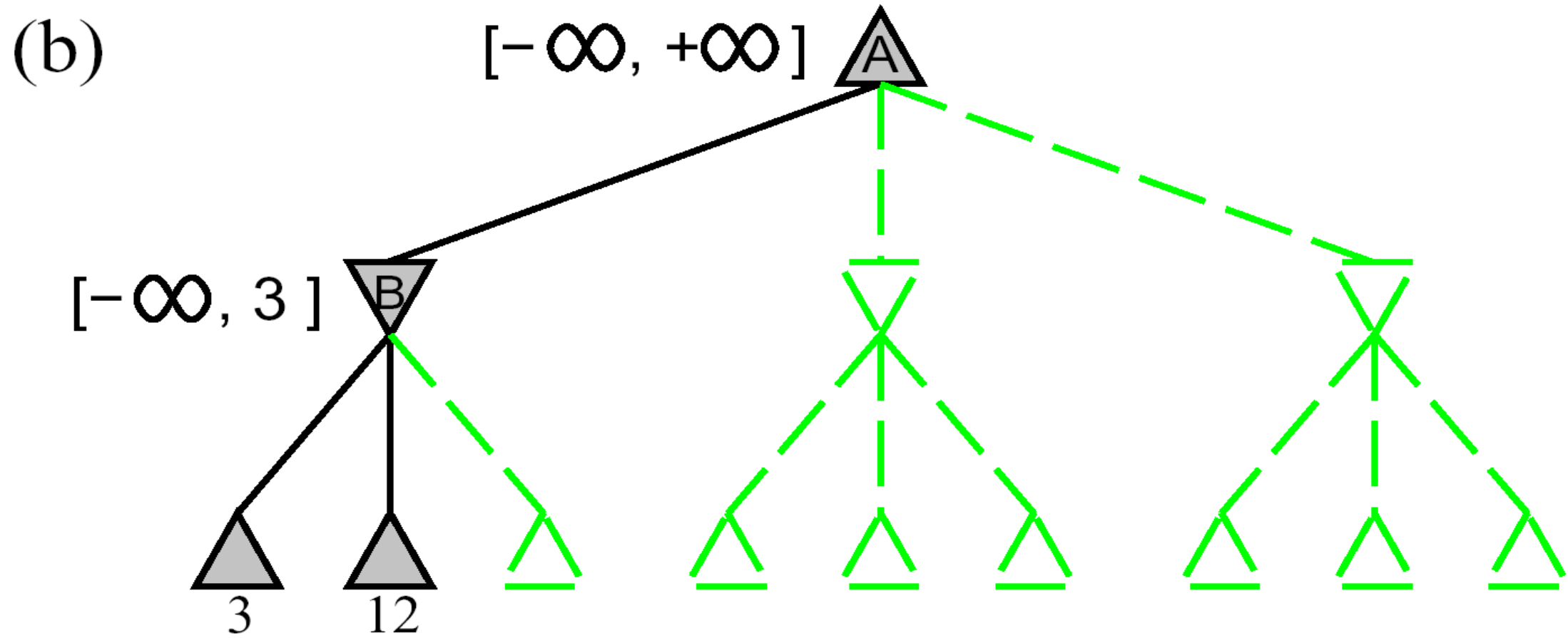
alpha-beta search

```
function ALPHA-BETA-DECISION(state) returns an action  
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))
```

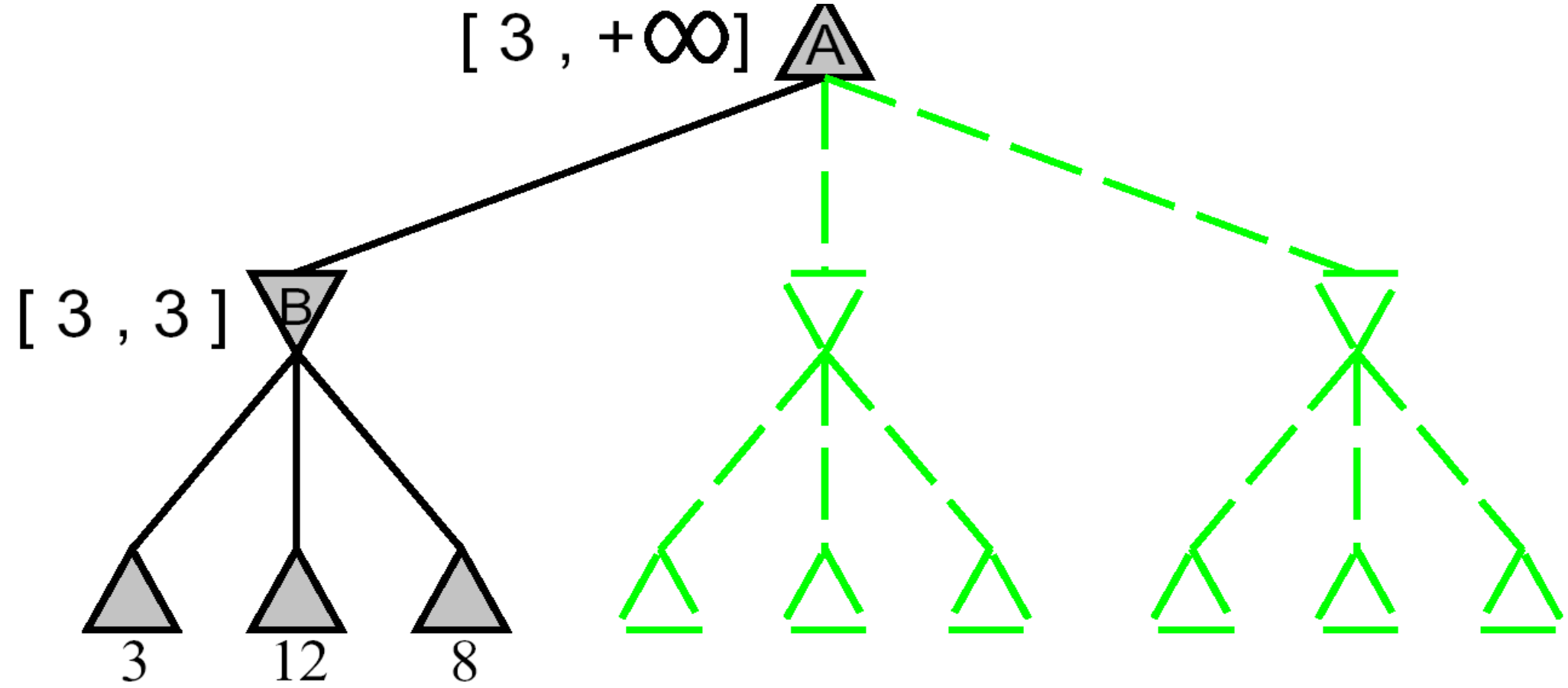
```
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  inputs: state, current state in game  
            $\alpha$ , the value of the best alternative for MAX along the path to state  
            $\beta$ , the value of the best alternative for MIN along the path to state  
  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for a, s in SUCCESSORS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$   
    if  $v \geq \beta$  then return v  
     $\alpha \leftarrow \text{MAX}(\alpha, v)$   
  return v
```

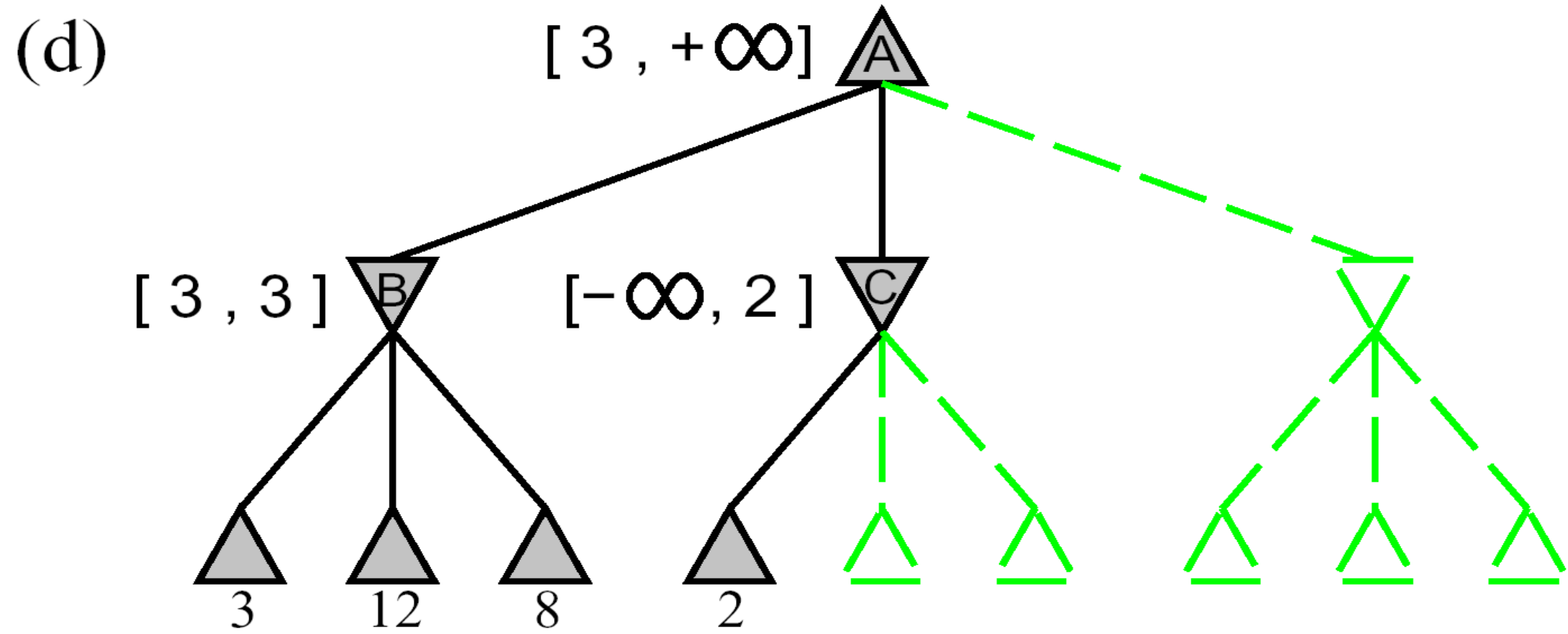
```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  same as MAX-VALUE but with roles of  $\alpha$ ,  $\beta$  reversed
```



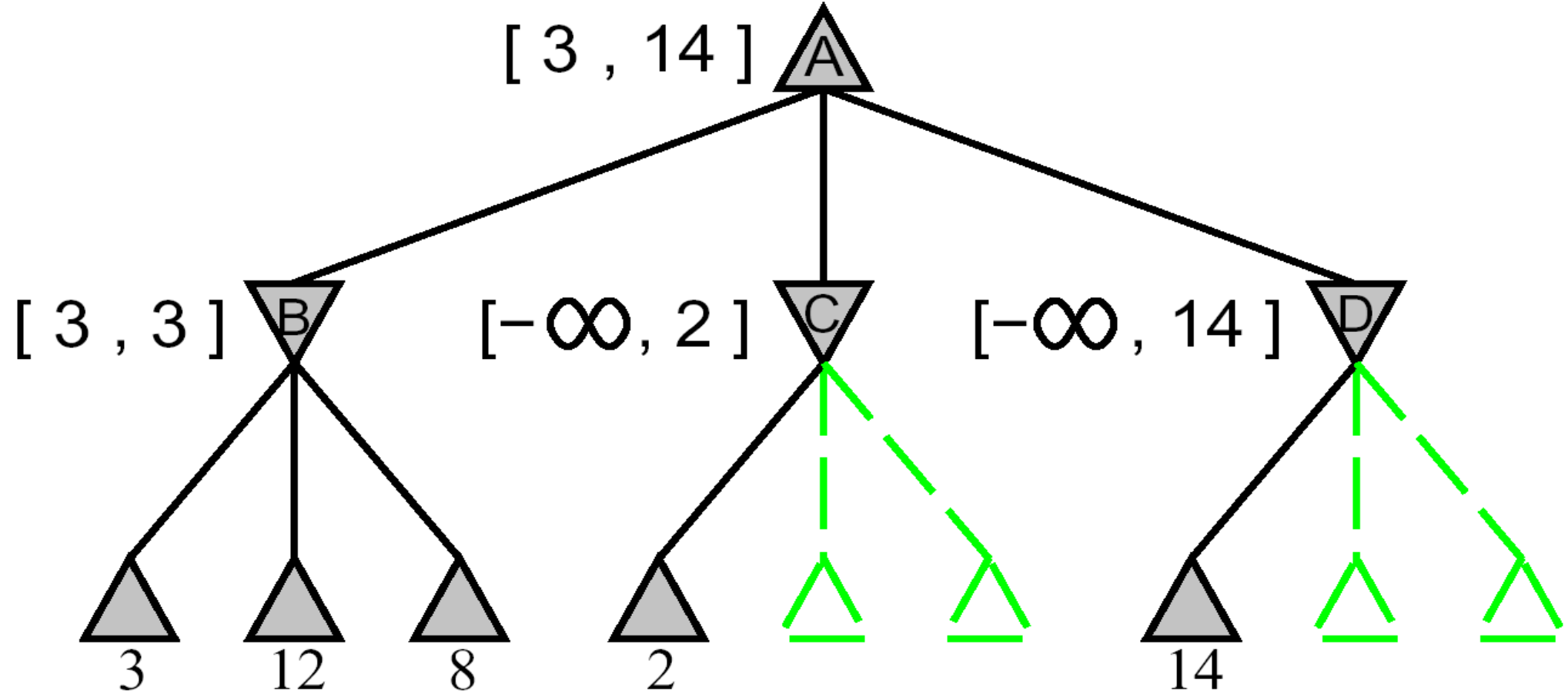


(c)

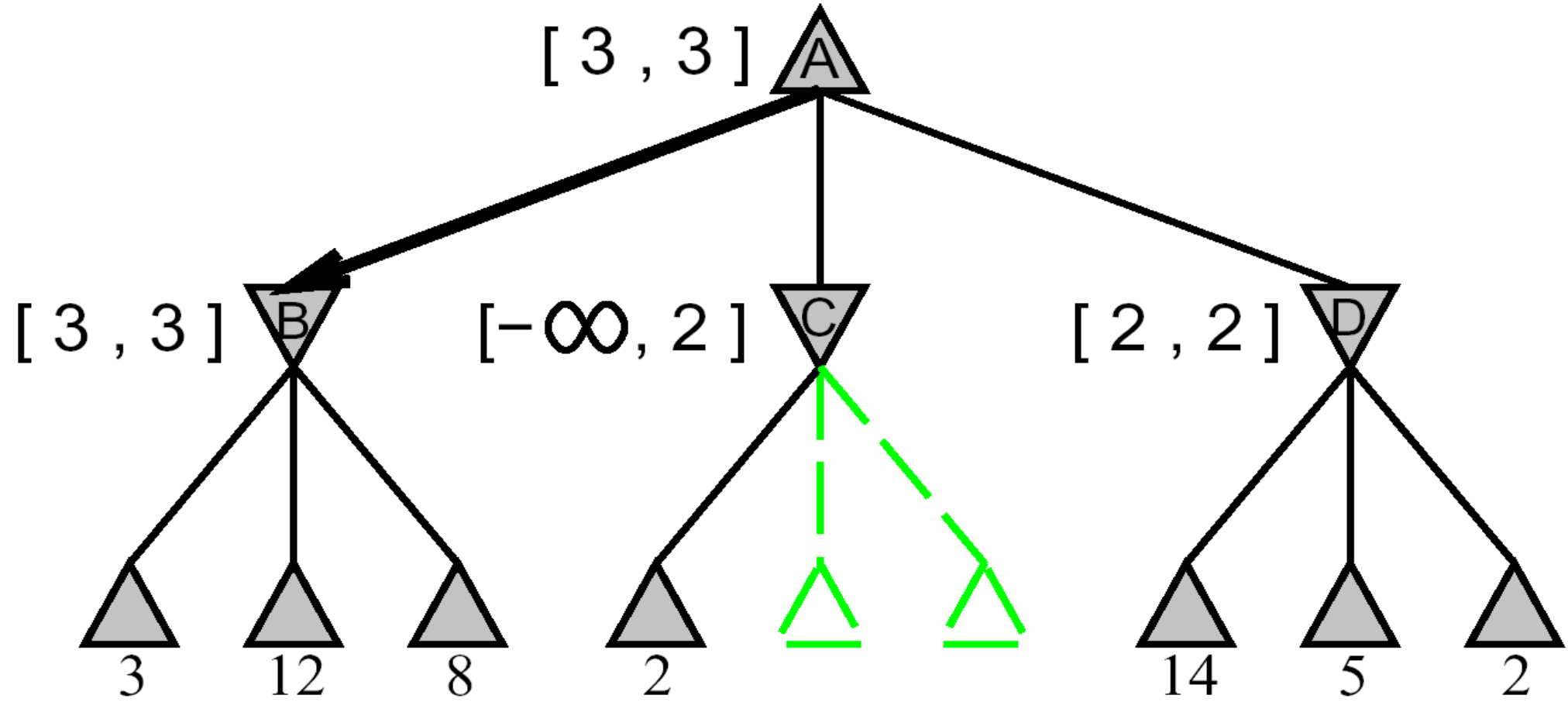




(e)



(f)



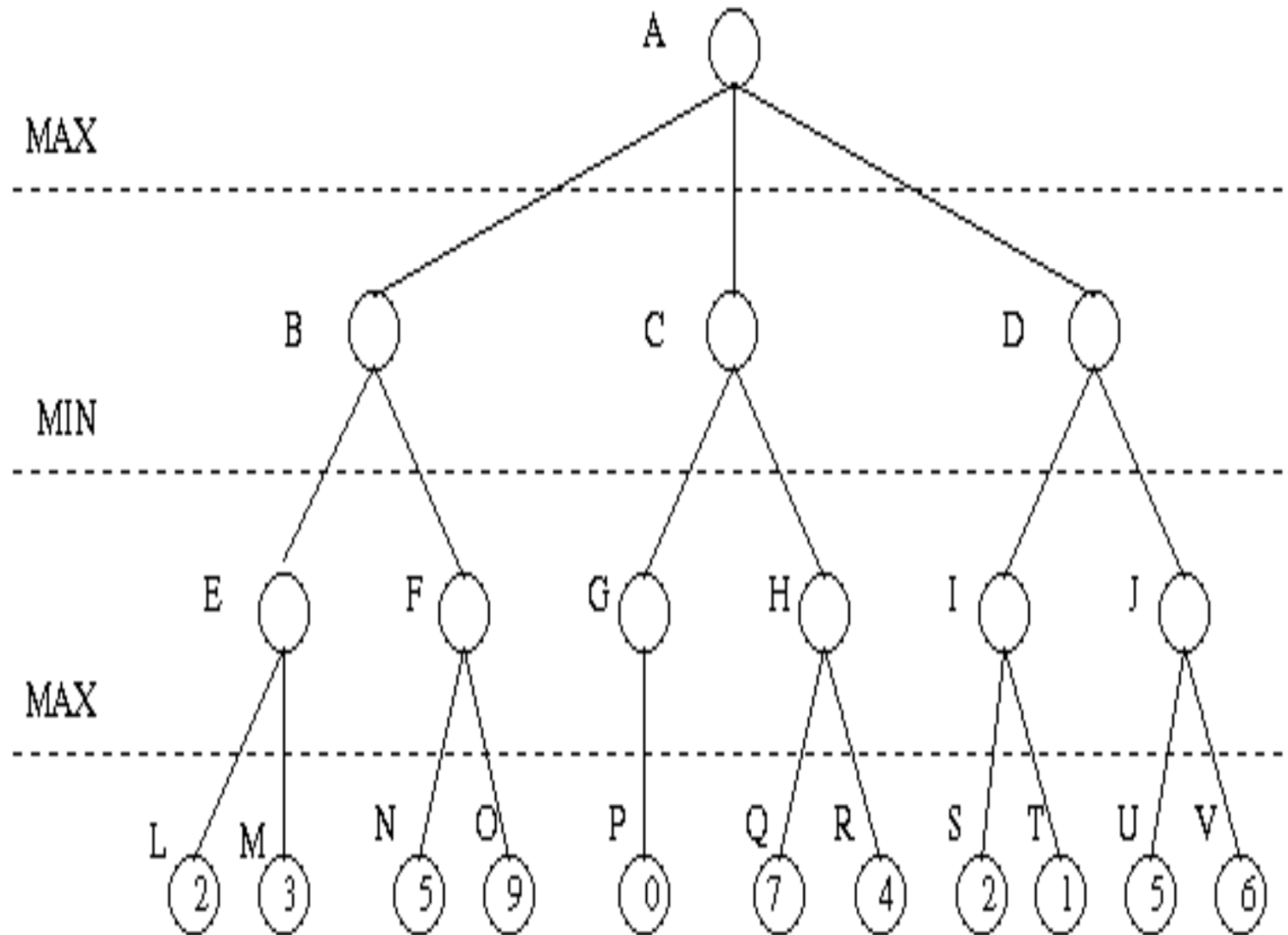
Rules of Thumb

- α is the best (highest) found so far along the path for **Max**
- β is the best (lowest) found so far along the path for **Min**
- Search below a **MIN node** may be alpha-pruned if the its $\beta \leq \alpha$ of some **MAX ancestor**.
- Search below a **MAX node** may be beta-pruned if the its $\alpha \geq \beta$ of some **MIN ancestor**.

Alpha-Beta Pruning Example

1. Search below a MIN node may be alpha-pruned if the beta value is $\leq$ to the alpha value of some MAX ancestor.

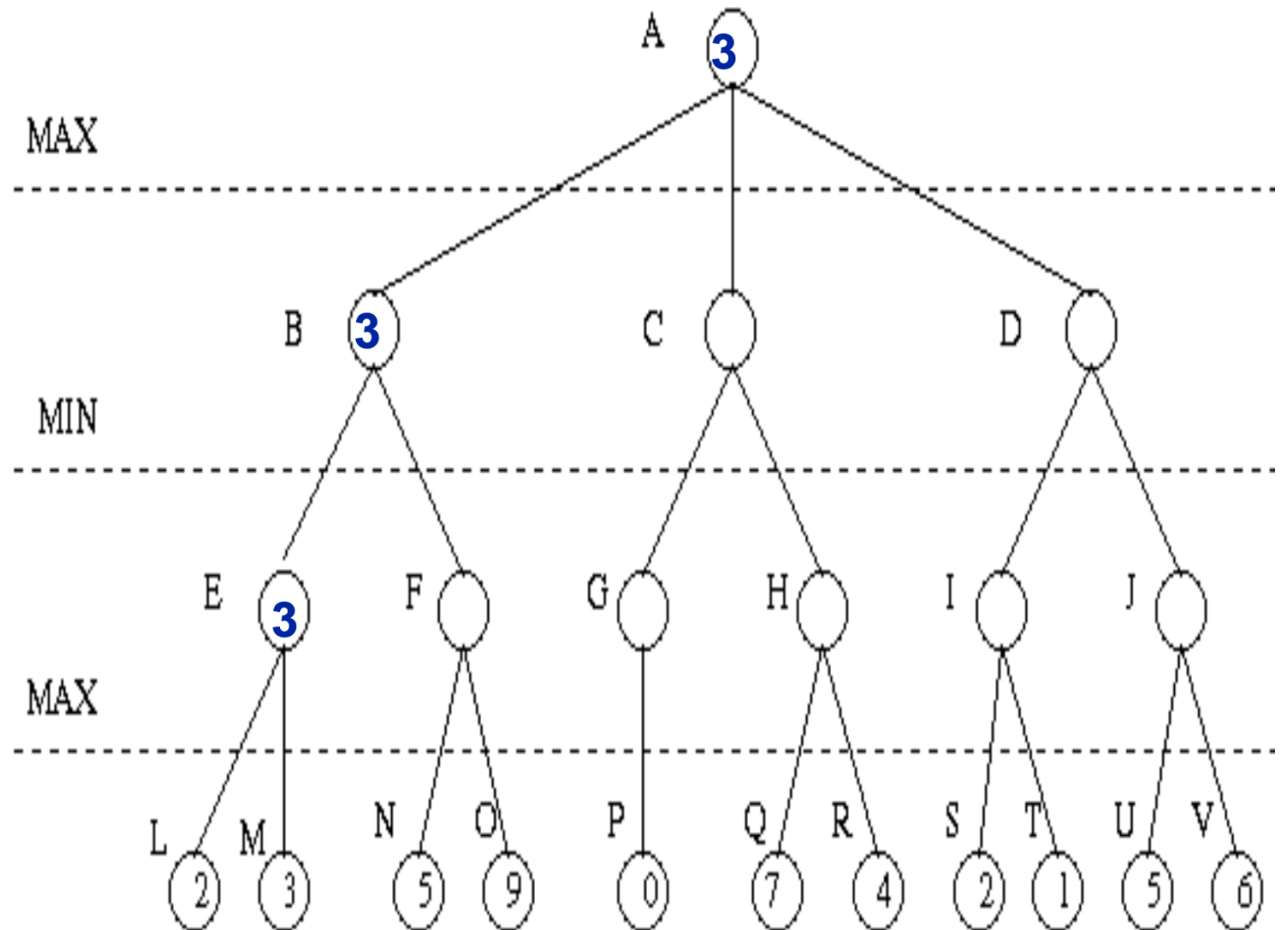
2. Search below a MAX node may be beta-pruned if the alpha value is $\geq$ to the beta value of some MIN ancestor.



Alpha-Beta Pruning Example

1. Search below a MIN node may be alpha-pruned if the beta value is $\leq$ to the alpha value of some MAX ancestor.

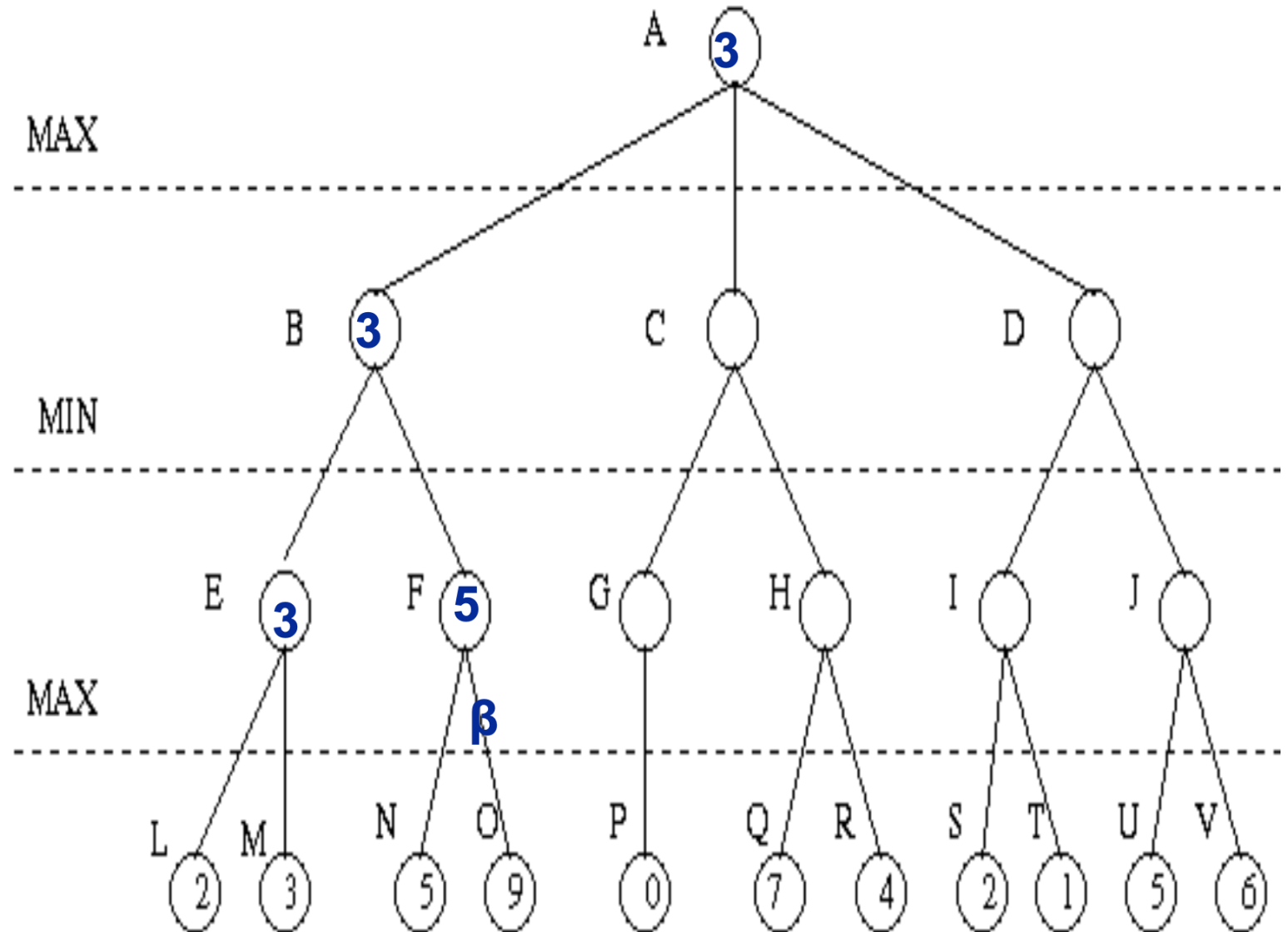
2. Search below a MAX node may be beta-pruned if the alpha value is $\geq$ to the beta value of some MIN ancestor.



Alpha-Beta Pruning Example

1. Search below a MIN node may be alpha-pruned if the beta value is $\leq$ to the alpha value of some MAX ancestor.

2. Search below a MAX node may be beta-pruned if the alpha value is $\geq$ to the beta value of some MIN ancestor.



Alpha-Beta Pruning Example

1. Search below a MIN node may be alpha-pruned if the beta value is $\leq$ to the alpha value of some MAX ancestor.

2. Search below a MAX node may be beta-pruned if the alpha value is $\geq$ to the beta value of some MIN ancestor.

